

Hermes Shopping Assistant Setup Guide

This is a pre-setup guide: get your machine ready before you start, so everything is installed and working ahead of time. It does not explain how any of this works internally. Treat it as a setup reference, not a walkthrough.

Repo: https://github.com/nihalkadri/Hermes_Shopping_Assistant_DataCamp.git

Two sections:

	Time	What it gets you
Section 1: Quick Setup (required)	~15 min	Run the assistant on your own machine
Section 2: Full Setup (optional)	~30–60 min	Deploy the same assistant to a free cloud VM, with a real Telegram bot and a real Shopify store

Section 1 alone is enough on its own. Do Section 2 if you want the full live version: reachable over Telegram Web in a browser, or later on the Telegram app on your phone too. Section 2 isn't time-sensitive: skip it for now and come back to it after the session if you'd rather.

Before you start: requirements

- **VS Code:** every command in this guide is run from its integrated terminal. Get it from code.visualstudio.com if you don't already have it.
- **Python 3.11 recommended** (3.10+ required, any version works; the `mcp` package fails outright on 3.8/3.9). Check: `python --version`. Windows: list all installed versions with `py -0p`. Don't have 3.11? Any 3.10+ version already on your machine is fine (e.g. `py -3.12`); otherwise get 3.11 from python.org/downloads. Tick **"Add Python to PATH"** during install.
- **Git:** only needed to download the code with `git clone`. The installer bundles Git Bash, but you don't need to use it; plain PowerShell (Windows) or Terminal (macOS/Linux) works fine for every command in this guide. Check: `git --version`. Missing? Install from git-scm.com/downloads.

That's it for prerequisites. Everything else (Hermes itself, Python packages) gets installed below.

Which terminal to use: every command in this guide, including the ones later that connect to a cloud VM, runs from VS Code's integrated terminal (**View** → **Terminal**, or `Ctrl+``) on your own machine. On Windows, set its shell to **PowerShell** using the dropdown at the top-right of the terminal panel, not Command Prompt; every Windows command in this guide is PowerShell syntax. On macOS/Linux, the default integrated terminal (bash/zsh) works as-is. A couple of steps below need two terminals open at once; they're labeled **Terminal A** and **Terminal B** when that happens.

SECTION 1: Quick Setup (required, ~15 min)

1.1 Clone the project

```
git clone https://github.com/nihalkadri/Hermes_Shopping_Assistant_DataCamp.git
cd Hermes_Shopping_Assistant_DataCamp
```

1.2 Create a virtual environment and install dependencies

Windows (PowerShell):

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

(No Python 3.11? Swap in any 3.10+ version you have, e.g. `py -3.10` or `py -3.12`.)

macOS / Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

PowerShell refuses to run the activate script? Run once, then retry: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

1.3 Install Hermes Agent

Use the exact command below, not the plain installer from Hermes's own docs. It pins the install to the specific version this whole guide (and the deployment in Section 2) was built and tested against. A plain install grabs whatever is newest that day, which can be a different version than what's covered here.

Windows (PowerShell):

```
& ([scriptblock]::Create((irm https://hermes-agent.nousresearch.com/install.ps1))) `
-Commit "56a41715dc3b8bf6f50a740ff9416c4036ef4259" `
-ForceCommit
```

macOS / Linux / WSL2:

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash -s -- \
  --commit 56a41715dc3b8bf6f50a740ff9416c4036ef4259 \
  --force-commit
source ~/.bashrc      # or ~/.zshrc on macOS
```

(`--force-commit` only matters if you already had Hermes installed from somewhere else before this guide; without it, the installer refuses to pin backward and you'd end up on a different version than everyone else. Harmless on a first install.)

Close and reopen your terminal, then confirm it installed and matches:

```
hermes --version
```

Should print **v0.20.1 (2026.8.13)**. Do not run `hermes update` afterward; that would move you off this pinned version and could introduce a mismatch with the rest of this guide.

1.4 Connect a free AI provider

```
hermes setup
```

Choose **Quick Setup (Nous Portal)**: a free OAuth login in your browser, no API key or credit card needed. This is the fastest and recommended path.

1.5 Verify it works

```
hermes
```

Type `hello, what can you do?` and confirm you get a real reply. Then type `/exit`.

1.6 Launch the dashboard once

```
hermes dashboard
```

This opens a local web UI and, on first launch, pulls down its own extra dependencies. Run it once now so that install happens ahead of time rather than the first time you try it. Confirm it opens in your browser at `127.0.0.1:9119`, then close the tab; you're done with it for now.

Section 1 is done.

The two steps below (start the backend, register the tool server) are optional to run now. Come back to them whenever you're ready to go further.

Optional: start the shop backend

Open Terminal B for this (a second VS Code terminal: click the + in the terminal panel, or `Ctrl+Shift+``) and leave it running there for the rest of this section:

```
python -m uvicorn shop_backend.main:app --reload --reload-dir shop_backend --port 8000
```

Verify it's up: open <http://localhost:8000/docs> in a browser.

Windows note: always use `python -m uvicorn`, never the bare `uvicorn` command; it can fail with "Fatal error in launcher" if this folder is ever moved.

Optional: register the shop tool server with Hermes

Back in Terminal A (Terminal B stays running the backend from the previous step).

Use the **full, absolute path** to your `.venv` Python. Hermes runs this as its own subprocess and won't see your terminal's active environment, so a bare `python` can resolve to the wrong interpreter entirely.

Windows:

```
hermes mcp add shop `
  --command `
  "C:\full\path\to\Hermes_Shopping_Assistant_DataCamp\.venv\Scripts\python.exe" `
  --args `
  "C:\full\path\to\Hermes_Shopping_Assistant_DataCamp\mcp_server\shop_mcp_server.py" `
  --env SHOP_API_URL=http://localhost:8000
```

macOS / Linux:

```
hermes mcp add shop \
  --command \
  "/full/path/to/Hermes_Shopping_Assistant_DataCamp/.venv/bin/python" \
  --args \
  "/full/path/to/Hermes_Shopping_Assistant_DataCamp/mcp_server/shop_mcp_server.py" \
  --env SHOP_API_URL=http://localhost:8000
```

Confirm it connected; should list 6 tools:

```
hermes mcp test shop
```

SECTION 2: Full Setup, Real-Time Simulation (optional, ~30–60 min)

This section puts the same assistant on a **permanently free** cloud VM, reachable over **Telegram** (Telegram Web in a browser, or the Telegram app on your phone later on), and wired up to a **real Shopify store** with a real (test) checkout. All of it costs **\$0/month**: every service used here has a genuine, non-trial free tier.

Do this in advance if you can. It involves account sign-ups that each take a few minutes to verify. If you don't get to it beforehand, that's fine: it's entirely optional, and there's nothing time-sensitive about it. Come back and do it after the live session, at your own pace, whenever you want the full experience.

How you'll talk to the VM: there's no separate app for this. Every command in this section, including the ones that end up running on the VM, is typed into a VS Code terminal on your own Windows machine (**View** → **Terminal**, same as Section 1). Step 2.5 opens a new one and calls it **Terminal C**; you `ssh` from it, and from that point whatever you type in that specific terminal window runs on the VM instead of your laptop, until you disconnect (`exit`). **Terminal A** is your regular local terminal (open a new one if you closed the one from Section 1). Every command block below says which one to use, and whether to open a new terminal or reuse one already open.

2.1 What you'll create accounts on (all free, no card charged)

- **Oracle Cloud:** hosts the bot 24/7. Sign-up requires a **real credit card** for identity verification, and a phone number for two-factor setup through the **Oracle Mobile Authenticator** app. The card is never charged; the shape used here is genuinely free forever, not a trial.
- **Telegram:** your bot's messaging platform
- **Shopify Partners:** a free development store with real inventory and a real (test-mode) checkout

An AI provider was already set up in Section 1.4 (Nous Portal); that's all this deployment needs. OpenRouter is just a backup the code can optionally fall back to if Nous ever hits a rate limit. It's not required and there's nothing to install for it here.

2.2 Get a Telegram bot token

1. Open Telegram, message [@BotFather](#) → send `/newbot` → pick a display name and a username ending in `bot` → copy the **token** it gives you.
2. Message [@userinfobot](#) → copy your **numeric** user ID (not your @username, the actual number).
3. Keep both values. You'll use the numeric ID twice in Step 2.8.

2.3 Set up a Shopify development store

1. Sign up at partners.shopify.com → **Stores** → **Add store** → **Development store** (free, no billing).
2. Once created, go to **Products** → **Import**, and upload `shopify_products_import_with_images.csv` from this repo. This adds 10 running-shoe products, with images and stock, in one step.
3. Go to **Settings** → **Payments** → activate **"(for testing) Bogus Gateway"**. This lets you complete real checkouts with fake test cards. No real money ever moves.
4. Note your store's domain: it looks like `something-random.myshopify.com` (visible in your browser URL bar inside the store admin, or under **Settings** → **Domains**).

2.4 Provision the Oracle Cloud Always Free VM

1. Sign up at cloud.oracle.com. Have a **real credit card** ready; Oracle asks for one during sign-up to verify your identity, and it is not charged. You'll also set up two-factor verification, typically through the **Oracle Mobile Authenticator** app (available for iOS/Android): install it and scan the QR code Oracle shows you when prompted. If Oracle offers you an SMS or email code instead, that works too; just complete whichever 2FA method it presents.
2. Create a **Compute instance**:
 - **Shape**: `VM.Standard.E2.1.Micro` (x86, 1 GB RAM). Use this one directly: it's what this whole guide is built and tested against, and it's part of Oracle's Always Free tier, genuinely free forever, not a trial.
 - **Image**: Canonical Ubuntu 24.04.
 - **Networking section of the same wizard**: a brand-new account has no VCN yet, so choose the option to create one; Oracle names it automatically (e.g. `vcn-20260812-2341`). Leave **"Automatically assign a public IPv4 address"** turned on; it's checked by default for a public subnet.
 - Generate an SSH key pair when prompted and **download the private key file immediately**. It cannot be retrieved again later. You'll use this file for every `ssh / scp` command below.
3. **After the instance is created**, open its **Networking** tab and confirm:
 - A **Public IPv4 address** is listed under **Primary VNIC**; copy it down, every step below refers to it as `<your-vm-public-ip>` or `<vm-ip>`. If it's missing: **Attached VNICs** → click the VNIC → **IP administration** tab → the ... (**Actions**) menu on the primary private IP → **Edit** → set **Public IP type** to **Ephemeral public IP** → **Update**.
 - Under **Quick actions**, run **"Connect public subnet to internet"** if it's offered there. It wires up an internet gateway, a network security group, and a route table in one step; without it the VM has no route out even with a public IP assigned. No inbound firewall rules are needed beyond SSH. Telegram polling means the bot calls out, nothing calls in.

2.5 Prepare the VM

VS Code terminal, open a new one (Terminal C), on your local Windows machine. SSH in from it; this connects that specific terminal to the VM, and everything else in this step (and in 2.7's second part, 2.8, and 2.9) runs in this same Terminal C, not your laptop:

```
ssh -i "path/to/your-downloaded-key.key" ubuntu@<your-vm-public-ip>
```

Add 2 GB of swap (required: 1 GB RAM alone isn't enough to build this, and the process gets killed without it):

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Install Docker:

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
```

Log out (`exit`) and SSH back in (same command as above, still in Terminal C) for the group change to apply.

2.6 Copy the project to the VM

Before copying, point the config at your own Shopify store. Open `hermes/cli-config.yaml` in your local project folder and, under `mcp_servers` → `shopify` → `env`, replace the `SHOPIFY_STORE_DOMAIN` value with the domain you noted in Step 2.3. It ships pointing at the demo's own store; without this edit, your bot would search and check out against that store instead of yours.

VS Code terminal, open a new one (Terminal A) on your local Windows machine (not Terminal C, which is busy being connected to the VM). This is the same folder you cloned in Section 1:

```
scp -r -i "path/to/your-key.key" `
"C:\path\to\Hermes_Shopping_Assistant_DataCamp" `
ubuntu@<vm-ip>:~/shop-bot
```

2.7 Copy your Nous login to the VM

The primary AI model runs through Nous Portal (same free login from Section 1.4). There's no browser available on a headless server to log in directly there, so you copy the login you already created on your own machine instead.

VS Code terminal, Terminal A (the same one from Step 2.6), on Windows:

```
scp -i "path/to/your-key.key" `
"$env:LOCALAPPDATA\hermes\auth.json" `
ubuntu@<vm-ip>:~/auth.json
```

macOS/Linux:

```
scp -i "path/to/your-key.key" ~/.hermes/auth.json ubuntu@<vm-ip>:~/auth.json
```

Switch to Terminal C (still connected to the VM from Step 2.5; reconnect with the `ssh` command from Step 2.5 if you'd closed it), and run:

```
mkdir -p ~/hermes-data
mv ~/auth.json ~/hermes-data/auth.json
chmod 600 ~/hermes-data/auth.json
```

If you registered any *other* AI providers locally beyond Nous, this copies those too. Harmless, just extra entries in the same file.

2.8 Build and run

VS Code terminal, Terminal C (still connected to the VM):

```
cd ~/shop-bot

docker build -t shopping-bot .

docker run -d --name shopping-bot \
  --restart unless-stopped \
  -v ~/hermes-data:/root/.hermes \
  -e TELEGRAM_BOT_TOKEN=...your-token-from-2.2... \
  -e TELEGRAM_ALLOWED_USERS=<your numeric telegram id> \
  -e TELEGRAM_HOME_CHANNEL=<same numeric id> \
  -e HERMES_TELEGRAM_DISABLE_FALLBACK_IPS=true \
  shopping-bot
```

If you did set up the optional OpenRouter backup, add `-e OPENROUTER_API_KEY=sk-or-...your-key...` to the command above. Without it, the bot just runs on Nous alone. The fallback code path checks for that variable and quietly does nothing if it's missing; it doesn't error or block startup.

The bind mount at `/root/.hermes` (the whole folder, not a subfolder) is what makes memory, sessions, and skills survive restarts, and (from Step 2.7) is also where your Nous login lives now.

The model configuration is already set correctly in this repo's `hermes/cli-config.yaml` . Nothing to edit there beyond the store domain from Step 2.6.

2.9 Verify

VS Code terminal, Terminal C (still connected to the VM):

```
sudo tail -f ~/hermes-data/logs/agent.log
```

(Use this, not `docker logs` : container output is buffered and lags badly without it. The log folder is root-owned since the container runs as root, hence `sudo` . Leave this running and switch away from Terminal C without closing it.)

Telegram app or web (not a VS Code terminal at all): message your bot:

```
Find Nike running shoes under $100, size 10
```

A message right after first starting the container can go unanswered. A one-time race between the bot connecting and the message arriving. Just send it again.

"Connecting to Telegram (attempt 1/8)" with no further log lines is normal, not stuck. That's just what an idle, successfully connected bot looks like while waiting for your next message.

2.10 Try the real Shopify checkout

Telegram app or web (not a VS Code terminal):

```
Using the shopify store tools, find running shoes under $100
Using the shopify store tools, add the first one to my cart
Using the shopify store tools, what's in my cart? Show the checkout URL.
```

Open the checkout link it gives you and complete it with the Bogus Gateway test card: **card number** `1` , any future expiry date, any 3-digit CVC, name on card **"Bogus Gateway"**. No real money moves. Check your Shopify admin's **Orders** page: the order will be there.

FAQ / Troubleshooting

Problem	Fix
<code>hermes: command not found</code> after installing	Close and reopen your terminal; on Linux/macOS also run <code>source ~/.bashrc</code>
<code>pip install -r requirements.txt</code> fails on the <code>mcp</code> package	You're on Python 3.9 or older; <code>mcp</code> requires 3.10+. Check with <code>python --version</code>
PowerShell won't run <code>Activate.ps1</code>	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , then retry
"Fatal error in launcher" running <code>uvicorn</code>	Always use <code>python -m uvicorn ...</code> , never the bare <code>uvicorn</code> command
<code>hermes mcp test shop</code> says disconnected	Re-run the <code>hermes mcp add</code> command (a failed connection saves the server as disabled); make sure the backend (Section 1's optional section) is running first
Hermes replies are empty or error out	Run <code>hermes doctor</code> for diagnostics, then <code>hermes model</code> to check your model config
Bot replies but never seems to use the shop tools	Check <code>hermes model</code> ; the configured model must support tool calling
Bot starts, logs look clean, never replies on Telegram	<code>TELEGRAM_HOME_CHANNEL</code> env var wasn't set when the container started; re-run the <code>docker run</code> command from Step 2.8 with it included
Bot was working, then stopped replying (or replies got slow) a day or so later	Your VM's Nous login has likely gone stale; that free login doesn't last forever once copied to a server. Run <code>scripts\refresh_nous_auth.ps1</code> (see "Right before the live session" below) to fix it
Bot forgets everything after a restart	The volume must be mounted at exactly <code>/root/.hermes</code> , not a subfolder of it
Docker build gets killed partway through	You skipped the 2 GB swap file in Step 2.5; add it, then rebuild
<code>ssh / scp</code> says <code>Permission denied (publickey)</code>	Double-check the key path and that you're connecting as <code>ubuntu@<vm-ip></code> , not another username. On macOS/Linux, the key file itself also needs restricted permissions: <code>chmod 600 path/to/your-key.key</code>
<code>ssh / scp</code> hangs or says connection refused/timed out	Revisit Step 2.4: check the instance has a VCN, a public IP under Primary VNIC, and that "Connect public subnet to internet" has been run
Can't reach the VM's public IP at all	Revisit Step 2.4: check the instance has a VCN, a public IP under Primary VNIC (or assign one via IP administration), and that "Connect public subnet to internet" has been run
Shopify catalog search returns nothing	In Shopify Admin, confirm the imported products are published to the Online Store sales channel and have available inventory greater than 0 ; zero-stock items don't appear in search at all

Problem	Fix
Shopify results/checkout look like someone else's store, not yours	You skipped the <code>SHOPIFY_STORE_DOMAIN</code> edit in Step 2.6; open <code>hermes/cli-config.yaml</code> , fix it under <code>mcp_servers</code> → <code>shopify</code> → <code>env</code> , then re-copy the project (Step 2.6) and rebuild (Step 2.8)
An in-stock-looking product still says "sold out" when adding to cart	A real Shopify behavior, not a bug. The catalog listing's "available" flag can lag behind the live cart-add inventory check. If your own testing exhausts a product's stock, restock it generously (10–20+ units) in Shopify Admin

Right before the live session

If you completed Section 2, refresh your VM's Nous login first. That free login can go stale after about a day once it's copied to a server, so do this close to when you'll actually use it, not days ahead.

Windows (PowerShell):

```
.\scripts\refresh_nous_auth.ps1 `
  -VmIp <your-vm-public-ip> `
  -SshKey "path/to/your-key.key"
```

This re-checks your local Nous login, copies a fresh one to the VM, and restarts the bot there. macOS/Linux: there's no script for this yet; repeat Step 2.7's `scp` and VM commands instead, same effect. Skip this whole step if you only did Section 1.

Then, re-clone the repo. A prompts file with the exact wording used during the live walkthrough gets added to this repo on the day of the session itself, so you can follow along and copy-paste each prompt as it's used rather than retyping it. Since it isn't there yet when you complete the setup above, pull the latest version shortly before the session starts:

```
git pull
```

If you edited `hermes/cli-config.yaml` in Step 2.6, `git pull` may warn about local changes instead of pulling cleanly. If that happens: `git stash`, then `git pull`, then `git stash pop` to bring your edit back.

If you cloned somewhere you can't easily find again, a fresh `git clone` of the same URL from the top of this guide works just as well.